



Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria en
Ingeniería y Tecnologías Avanzadas

Ingeniería Telemática

Base de Datos Distribuidas

3TM2

Carlos De La Cruz Sosa

Práctica 4

Integrantes:

García Ortiz Martha Lesly

Hernández Arteaga Itzel

24 de septiembre de 2026

Problema 1: Lectura sucia

La lectura sucia es un problema de concurrencia que ocurre cuando una transacción consulta información modificada por otra transacción que todavía no ha confirmado sus cambios. Si la transacción que realizó la modificación ejecuta posteriormente un ROLLBACK, los datos leídos por la otra transacción no corresponden a un estado confirmado de la base de datos.

Desarrollo

Se trabajó con dos tablas existentes: Production.Product, que contiene información de los productos, y Production.ProductInventory, que almacena sus existencias por ubicación.

Se seleccionó el producto Bearing Ball, con identificador ProductID = 2, y su registro de inventario correspondiente a LocationID = 50 para realizar la transacción correspondiente.

Los valores iniciales fueron un precio de lista de \$0.00 y 364 unidades en inventario.

Se utilizaron dos ventanas de consulta para representar dos transacciones concurrentes.

```
BEGIN TRANSACTION;

UPDATE Production.ProductInventory
SET Quantity = 400
WHERE ProductID = 2
    AND LocationID = 50;

WAITFOR DELAY '00:00:20';
ROLLBACK TRANSACTION;
```

Fig.1 Transacción explícita

ProductID	NombreProducto	Precio	LocationID	Existencias
1	Adjustable Race	0,00	1	408
1	Adjustable Race	0,00	6	324
1	Adjustable Race	0,00	50	353
2	Bearing Ball	0,00	1	427
2	Bearing Ball	0,00	6	318
2	Bearing Ball	0,00	50	364
3	BB Ball Bearing	0,00	1	585
3	BB Ball Bearing	0,00	6	443
3	BB Ball Bearing	0,00	50	324
4	Headset Ball B...	0,00	1	512

Fig.2 Resultados de la consulta

Escenario 1. READ UNCOMMITTED

La transacción A modificó el precio del producto a \$100.00 y sus existencias a 400 unidades, sin confirmar los cambios.

Posteriormente, la transacción B ejecutó una consulta utilizando el nivel de aislamiento READ UNCOMMITTED y obtuvo los valores modificados: \$100.00 y 400 unidades.

Finalmente, A ejecutó ROLLBACK, cancelando ambas modificaciones. Los valores regresaron a \$0.00 y 364 unidades.

Este resultado demostró que B había leído información no confirmada que posteriormente fue descartada.

```
BEGIN TRANSACTION;

-- Modificar temporalmente el precio
UPDATE Production.Product
SET ListPrice = 100.00
WHERE ProductID = 2;

-- Modificar temporalmente las existencias
UPDATE Production.ProductInventory
SET Quantity = 400
WHERE ProductID = 2
AND LocationID = 50;
```

Fig.3 Modificación del producto Bearing Ball

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

SELECT
    P.ProductID,
    P.Name AS NombreProducto,
    P.ListPrice AS Precio,
    I.LocationID,
    I.Quantity AS Existencias
FROM Production.Product AS P
INNER JOIN Production.ProductInventory AS I
    ON P.ProductID = I.ProductID
WHERE P.ProductID = 2
    AND I.LocationID = 50;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Fig.4 Consulta en ventana B utilizando UNCOMMITTED

	ProductID	NombreProducto	Precio	LocationID	Existencias
1	2	Bearing Ball	100,00	50	400

Fig.5 Resultados de la ventana A y B

Resultados		Mensajes			
	ProductID	NombreProducto	Precio	LocationID	Existencias
1	2	Bearing Ball	0,00	50	364

Fig.6 Resultados de la ventana A y B después de ejecutar ROLLBACK TRANSACTION

Escenario 2. READ COMMITTED

Se repitieron las modificaciones de A, pero esta vez B utilizó el nivel de aislamiento READ COMMITTED.

Al ejecutar la consulta, B quedó esperando mientras A mantenía abierta su transacción.

Después de que A ejecutó ROLLBACK, B pudo continuar y obtuvo los valores originales: \$0.00 y 364 unidades.

El comportamiento observado corresponde al uso de bloqueos tradicionales, ya que la opción READ_COMMITTED_SNAPSHOT de la base de datos estaba desactivada.

```

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SET LOCK_TIMEOUT 60000;

BEGIN TRANSACTION;

SELECT
    P.ProductID,
    P.Name AS NombreProducto,
    P.ListPrice AS Precio,
    I.LocationID,
    I.Quantity AS Existencias
FROM Production.Product AS P
INNER JOIN Production.ProductInventory AS I
    ON P.ProductID = I.ProductID
WHERE P.ProductID = 2
    AND I.LocationID = 50;
COMMIT TRANSACTION;
SET LOCK_TIMEOUT -1;

```

Fig.7 Consulta en ventana B utilizando COMMITTED



Fig.8 Resultado de COMMITTED

Análisis de resultados

La diferencia entre ambos escenarios estuvo en el nivel de aislamiento utilizado por la transacción B.

Con READ UNCOMMITTED, B pudo consultar los valores modificados por A antes de que fueran confirmados. Esto produjo una lectura sucia.

Con READ COMMITTED, B no pudo leer las modificaciones pendientes y tuvo que esperar a que A finalizara su transacción.

Al ejecutarse ROLLBACK, se cancelaron los cambios y B obtuvo los valores originales.

Conclusiones

Se comprobó que una lectura sucia puede ocurrir cuando una transacción consulta información modificada por otra transacción que todavía no ha confirmado sus operaciones.

Se concluye que el nivel de aislamiento influye en la consistencia de la información consultada y en el comportamiento de las transacciones concurrentes.

Problema 2. Lectura no repetible

La lectura no repetible es un problema de concurrencia que puede ocurrir cuando una transacción consulta información mientras otra transacción modifica los datos utilizados en el análisis.

Como consecuencia, la primera transacción puede combinar valores correspondientes a diferentes momentos de la base de datos, aunque cada valor consultado haya sido confirmado.

Desarrollo

Se trabajó con las tablas Production.Product y Production.ProductInventory, utilizando el producto Bearing Ball, identificado mediante ProductID = 2, y su registro de inventario correspondiente a LocationID = 50.

Los valores iniciales fueron un precio de lista de \$0.00 y 364 unidades en inventario.

Se utilizaron dos ventanas de consulta para representar dos transacciones concurrentes.

Escenario 1. READ COMMITTED

La transacción A inició un análisis utilizando el nivel de aislamiento READ COMMITTED y consultó el precio del producto, obteniendo \$0.00.

Posteriormente, la transacción B modificó el precio a \$100.00 y las existencias a 400 unidades. Ambas modificaciones fueron confirmadas mediante COMMIT.

Después, A continuó su análisis y consultó las existencias del producto, obteniendo 400 unidades.

El resultado demostró que A combinó el precio anterior con las existencias nuevas, obteniendo información correspondiente a dos estados diferentes de la base de datos.

```

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

DECLARE @Count INT;
DECLARE @Total DECIMAL(19, 4);
DECLARE @Average DECIMAL(19, 4);
DECLARE @CalculatedAverage DECIMAL(19, 4);

BEGIN TRANSACTION;

SELECT @Count = COUNT(*)
FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43661;

SELECT
    'Cantidad inicial' AS Paso,
    @Count AS NumeroDetalles;

WAITFOR DELAY '00:00:20';

SELECT @Total =
    SUM(
        CAST(UnitPrice AS DECIMAL(19, 4))
        * OrderQty
        * (1 - CAST(UnitPriceDiscount AS DECIMAL(19, 4)))
    )

```

Fig.9 Conteo de ordenes

```

SELECT
    'Total intermedio' AS Paso,
    @Total AS TotalPedido;

WAITFOR DELAY '00:00:20';

SELECT @Average =
    AVG(
        CAST(UnitPrice AS DECIMAL(19, 4))
        * OrderQty
        * (1 - CAST(UnitPriceDiscount AS DECIMAL(19, 4)))
    )
FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43661;

SET @CalculatedAverage =
    @Total / NULLIF(@Count, 0);

SELECT
    @Count AS NumeroDetallesInicial,
    @Total AS TotalIntermedio,
    @Average AS PromedioActual,
    @CalculatedAverage AS PromedioCalculado,
    @Average - @CalculatedAverage AS Diferencia;

```

Fig.10 Proceso 2 de ventana A

The screenshot shows a SQL Server query result window with a zoom level of 122%. The window has tabs for 'Resultados' (Results) and 'Mensajes' (Messages). The 'Resultados' tab is active, displaying a table with the following data:

	SalesOrderDetailID	OrderQty	UnitPrice	UnitPriceDiscount
1	15	10	809.76	0.00

Fig.11 Resultado de la modificación de la ventana B

Escenario 2. REPEATABLE READ

Se restauraron los valores originales del producto y se repitió el experimento utilizando REPEATABLE READ.

En esta ocasión, la transacción A consultó inicialmente tanto el precio como las existencias, obteniendo \$0.00 y 364 unidades.

Cuando B intentó modificar los registros, quedó esperando debido a los bloqueos mantenidos por A.

Al repetir la consulta, A obtuvo nuevamente los mismos valores: \$0.00 y 364 unidades.

Finalmente, A ejecutó COMMIT y liberó sus bloqueos. B pudo continuar y confirmó sus modificaciones, estableciendo el precio en \$100.00 y las existencias en 400 unidades.

```

SELECT @Average =
    AVG(
        CAST(UnitPrice AS DECIMAL(19, 4))
        * OrderQty
        * (1 - CAST(UnitPriceDiscount AS DECIMAL(19, 4)))
    )
FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43661;

SET @CalculatedAverage =
    @Total / NULLIF(@Count, 0);

SELECT
    @Count AS NumeroDetallesInicial,
    @Total AS TotalIntermedio,
    @Average AS PromedioActual,
    @CalculatedAverage AS PromedioCalculado,
    @Average - @CalculatedAverage AS Diferencia;

```

```

BEGIN TRANSACTION;

UPDATE Sales.SalesOrderDetail
SET OrderQty = 10
WHERE SalesOrderDetailID = (
    SELECT MIN(SalesOrderDetailID)
    FROM Sales.SalesOrderDetail
    WHERE SalesOrderID = 43661
);

COMMIT TRANSACTION;

SELECT TOP (1)
    SalesOrderDetailID,
    OrderQty,
    UnitPrice,
    UnitPriceDiscount
FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43661
ORDER BY SalesOrderDetailID;

```

Fig.12 Consulta de precio y existencias en ventana A

Fig.13 Modificación del producto, precio y existencia en ventana B

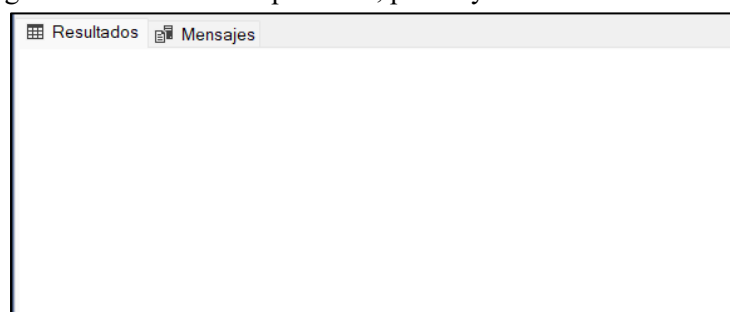


Fig.14 Respuesta de la modificación en ventana B

	SalesOrderDetailID	OrderQty	UnitPrice	UnitPriceDiscount
1	15	10	809.76	0.40

Fig.15 Resultados después de ejecutar COMMIT TRANSACTION; en ventana A

Análisis de resultados

En el primer escenario, READ COMMITTED permitió que A observara datos confirmados correspondientes a diferentes momentos de la base de datos. Esto produjo un análisis inconsistente.

En el segundo escenario, REPEATABLE READ mantuvo protegidos los registros previamente leídos por A, impidiendo que B los modificara antes de que A finalizara su transacción.

De esta manera, A pudo repetir la consulta y obtener los mismos valores.

Conclusiones

Se comprobó que READ COMMITTED evita las lecturas sucias, pero no garantiza que los datos consultados en distintos momentos de una transacción permanezcan sin cambios.

Se concluye que el nivel de aislamiento influye tanto en la consistencia de los datos consultados como en la concurrencia de las operaciones. La elección del nivel adecuado depende de las garantías que requiere cada análisis.

Problema 3. Lectura fantasma

La lectura fantasma ocurre cuando una transacción ejecuta una consulta sobre un conjunto de filas y, antes de finalizar, otra transacción inserta una nueva fila que cumple con la misma condición de búsqueda.

Cuando la primera transacción repite la consulta, aparece una fila adicional que no estaba presente en la primera lectura. A esta nueva fila se le conoce como fila fantasma.

```
-- VENTANA A
BEGIN TRANSACTION;

DECLARE @ProductID INT = 707;

-- Primera lectura
SELECT
    COUNT(*) AS TotalFilasAntes
FROM Sales.SalesOrderDetail
WHERE ProductID = @ProductID;

SELECT TOP (10)
    SalesOrderID,
    SalesOrderDetailID,
    ProductID,
    OrderQty,
    UnitPrice,
    UnitPriceDiscount,
    CarrierTrackingNumber
FROM Sales.SalesOrderDetail
WHERE ProductID = @ProductID
ORDER BY SalesOrderDetailID DESC;
-- Retardo para que B intente insertar
WAITFOR DELAY '00:00:10';
```

Figura16.Ventana A

```
...glesi (84))" X [TAS.sql - LE...glesi (54))"
-- VENTANA B
WAITFOR DELAY '00:00:05';
SET LOCK_TIMEOUT 10000;

BEGIN TRY
    BEGIN TRANSACTION;

    DECLARE @ProductID INT = 707;
    DECLARE @SalesOrderID INT;
    DECLARE @SpecialOfferID INT;
    DECLARE @UnitPrice MONEY;
    DECLARE @UnitPriceDiscount MONEY;

    SELECT TOP (1)
        @SalesOrderID = SalesOrderID,
        @SpecialOfferID = SpecialOfferID,
        @UnitPrice = UnitPrice,
        @UnitPriceDiscount = UnitPriceDiscount
    FROM Sales.SalesOrderDetail
    WHERE ProductID = @ProductID
    ORDER BY SalesOrderDetailID;

    INSERT INTO Sales.SalesOrderDetail
    (
        SalesOrderID,
        CarrierTrackingNumber,
        OrderQty,
        ProductID,
        UnitPrice,
        UnitPriceDiscount
    )
    VALUES
    (
        @SalesOrderID,
        NULL,
        1,
        @ProductID,
        @UnitPrice,
        @UnitPriceDiscount
    )

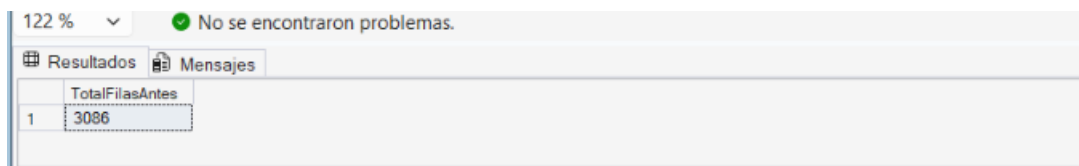
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
END CATCH
```

Fig17.Ventana B

Desarrollo del experimento

Para llevar a cabo el problema se utilizó SQL Server Management Studio y la base de datos AdventureWorks.

Se trabajó con las tablas Production.Product y Production.ProductInventory, utilizando el producto Bearing Ball con ProductID = 2.



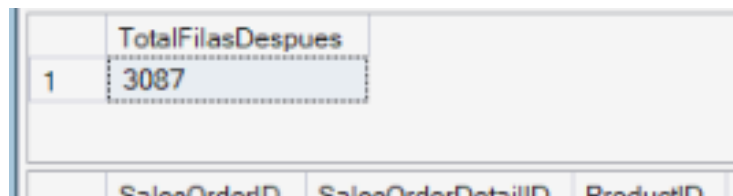
122 % No se encontraron problemas.

	TotalFilasAntes
1	3086

Fig18. Antes de provocar la lectura fantasma

En la primera prueba, la transacción A utilizó el nivel de aislamiento READ COMMITTED y consultó las filas de inventario existentes para el producto. Después, la transacción B insertó una nueva fila de inventario para el mismo producto y confirmó la operación mediante COMMIT.

Cuando la transacción A repitió la consulta dentro de la misma transacción, obtuvo una fila adicional. Esta fila representa la lectura fantasma, ya que no aparecía en la primera lectura, pero sí apareció en la segunda.



	TotalFilasDespues
1	3087

Fig19. Incremento de fila después de provocar la lectura fantasma

Conclusiones

Se comprobó que READ COMMITTED permite que aparezcan nuevas filas confirmadas entre dos consultas realizadas dentro de una misma transacción. Esto provocó una lectura fantasma.